F4RM: Feature-Restricted Round-Robin Random Mapping for Weightless Neural Networks

Anonymous Author(s)

Affiliation anonymized for double-blind review

Abstract. Weightless Neural Networks (WNNs) classify by reading bit-addressed entries from small lookup tables, replacing the multiply-accumulate operations of deep neural networks (DNNs) with a handful of memory reads. This makes them attractive for Edge AI deployment on memory- and energy-constrained devices, but the canonical WNN architecture, WiSARD, has left one structural component largely unexamined for four decades: the mapping from input bits to RAM positions, defaulted to a uniformly random permutation. We show that random mapping leaves measurable accuracy on the table whenever the input has exploitable structure, and that thermometer-encoded continuous features, the de-facto WiSARD binarization, have exactly such structure. We propose F4RM (Feature-Restricted Round-Robin Random Mapping), a heuristic mapping rule that maximizes per-RAM feature coverage at zero training cost and no added parameters. Two compositional extensions, F4RMS, an ensemble of independently-seeded F4RM WiSARDs, and F4RM-Cuckoo, which pairs F4RM with cuckoo-filter storage, demonstrate F4RM's composability with established WiSARD techniques. Across 21 tabular classification datasets with 200 paired evaluations per dataset and Bonferroni-corrected significance, F4RM beats random mapping on 13 of 21 datasets and never significantly loses; F4RMS extends this to 20 of 21. Across a 10-baseline Pareto-frontier comparison, F4RM-Cuckoo achieves Pareto-frontier membership on every one of the 21 datasets and a higher accuracy-per-byte ratio than every gradient-boosting baseline and random forest on every dataset evaluated. These results establish weightless networks with structured mapping as a competitive option for tabular classification at the extreme edge.

Keywords: Weightless neural networks · WiSARD · Tabular classification · Edge AI · Pareto evaluation

1 Introduction

Modern machine learning is increasingly deployed on devices that cannot afford it. Smart sensors, wearables, microcontrollers, and embedded health monitors, the "extreme edge" of computing, are projected to number in the tens of billions by the end of the decade [6], and most of them have less than a megabyte

of memory and a milliwatt-scale energy budget. Deep neural networks (DNNs), engineered for cloud-scale GPUs, do not transfer cleanly to this regime: even after aggressive quantization and pruning, their multiply-accumulate cost remains the dominant bottleneck on hardware where multiplication itself is expensive [19]. The result is a growing gap between where machine learning is most accurate and where it is most needed.

Weightless Neural Networks (WNNs) are a structurally different answer. Instead of computing weighted sums, WNNs classify by reading bit-addressed entries from a bank of small lookup tables; training is a single pass that sets table entries, inference is a handful of memory reads, and no floating-point arithmetic is required at any stage [1]. Recent work has pushed WNNs onto FPGAs and milliwatt-scale microcontrollers with accuracy competitive with quantized DNNs at orders-of-magnitude lower energy and latency [25,24,3]. WNNs are not a universal replacement for DNNs, but for tabular classification at the edge, wearable physiology, industrial sensors, low-cost embedded diagnostics, they are an attractive operating point.

The canonical WNN architecture, WiSARD [1], partitions a binary input vector into disjoint chunks of size a, routes each chunk to a Random-Access-Memory node that stores a learned response, and aggregates per-class responses to predict. The assignment of input bits to RAM positions, the *mapping*, is fixed at initialization, shared across all class discriminators, and in every standard implementation drawn as a uniformly random permutation. Forty years of WiSARD research have produced substantial improvements in storage [21,4], encoding [17], ensembling [7,24,18], bleaching [12], and most recently learnable mappings via differentiable relaxation [3]. The non-learned random mapping itself, however, has remained the default. Guarisa [13] surveyed mapping techniques in WiSARD, comparing random-shared, random-independent, stochastic-search, genetic, and entropy-based variants, but treated the mapping as a hyperparameter to optimize from data, not as a structural design decision tied to the input representation.

We argue that random mapping leaves measurable accuracy on the table whenever the input has exploitable structure, and that thermometer-encoded continuous features, the de-facto binarization in modern WiSARD work [17], have exactly such structure. The t bits of a thermometer-encoded feature form a contiguous block of 1s followed by 0s, so they admit only $t + 1$ distinct values instead of 2^t. When random mapping places multiple bits from the same feature into the same RAM, the redundancy collapses the RAM's effective address space; meanwhile, another RAM loses the chance to read that feature at all. Both costs are invisible per-RAM but consistent in aggregate.

We introduce **F4RM** (Feature-Restricted Round-Robin Random Mapping), a heuristic mapping rule motivated by this redundancy structure. F4RM constrains the mapping so that each RAM reads from the maximum possible number of distinct features, while preserving the random selection of *which* thermometer bit of each feature contributes. The construction runs once at initialization, has no data dependency on the training set, adds no parameters, and changes neither

training nor inference of the underlying WiSARD. F4RM is, to our knowledge, the first WiSARD mapping rule motivated structurally, by a property of the input encoding, rather than optimized through search.

To assess whether F4RM composes with the rest of the WiSARD ecosystem, we further evaluate two natural extensions. **F4RMS** (F4RM enSemble) trains K independently-seeded F4RM WiSARDs and aggregates their per-class responses by summation, inheriting established WiSARD ensembling practice [24,18] while compounding F4RM's per-model benefit. **F4RM-Cuckoo** replaces dense RAM storage with cuckoo filters [9,2], demonstrating that F4RM's accuracy gain is preserved under memory-efficient storage backends introduced in prior WiSARD work. Neither extension is itself a novel architecture; both serve to confirm that F4RM is orthogonal to and composable with the rest of the WiSARD literature.

Our contributions are:

1. **F4RM**, a heuristic feature-coverage mapping rule for WiSARD, motivated by the redundancy structure of thermometer encoding. The rule identifies two costs of random mapping (feature-coverage loss and effective-capacity loss) and yields a falsifiable capacity-decay prediction that we confirm empirically. F4RM requires no parameters, no training overhead, and no hyperparameters beyond standard WiSARD.
2. **Two compositional extensions**, F4RMS and F4RM-Cuckoo, demonstrating that F4RM combines cleanly with established WiSARD ensembling and cuckoo-filter storage. Neither extension is a novel architecture in itself.
3. **Empirical validation** on 21 tabular classification datasets: F4RM beats random mapping at matched hyperparameters on 13 of 21 with Bonferroni-corrected significance and never significantly loses; F4RMS extends to 20 of 21. Across a 10-baseline Pareto-frontier comparison, F4RM-Cuckoo achieves Pareto-frontier membership on every one of 21 datasets and a higher accuracy-per-byte ratio than every gradient-boosting baseline and random forest on every one of 21 datasets. Notebooks: `https://anonymous.4open.science/r/f4rm-notebook/notebooks/f4rm/`; `wisardpkg` implementation: `https://anonymous.4open.science/r/f4rm-implementation/wisardpkg/`.

The paper is divided as follows: Section 2 surveys prior work along the axes of mapping, memory, and ensembling; Section 3 defines F4RM, develops its mechanistic justification, and introduces the F4RMS and F4RM-Cuckoo extensions (Section 3.5); Section 4 describes the experimental methodology; Section 5 reports results; and Section 6 discusses limitations and concludes.

2 Related Work

We organize related work along the three axes most relevant to F4RM: mapping (Section 2.1), where F4RM contributes; storage backends (Section 2.2), where F4RM-Cuckoo composes; and ensembling (Section 2.3), where F4RMS inherits established practice.

2.1 Mapping

The original WiSARD [1] introduced a uniformly random bit-to-RAM permutation and four decades of subsequent work have largely preserved the choice. The mapping is convenient: it requires no design effort, introduces no bias toward any particular feature interpretation, and matches the default in every reference implementation. But its convenience has discouraged scrutiny, even as the surrounding architecture has been substantially refined.

Two prior efforts have departed from random mapping. Guarisa [13] conducted a comparative study of mapping techniques in WiSARD, evaluating random-shared, random-independent, stochastic-search, genetic, and entropy-clustering variants, each treating the mapping as a hyperparameter to optimize from data via training-set statistics or stochastic search. Differentiable WiSARD [3] makes the mapping a learnable parameter trained via a differentiable relaxation, achieving substantial accuracy gains at the cost of replacing WiSARD's single-pass training with gradient-based optimization. F4RM occupies a different point: a deterministic mapping rule motivated by the structure of thermometer encoding, requiring no data, no search, and no optimization. BTHOWeN's H3 hash routing [25] is a non-random alternative motivated by memory efficiency rather than input structure.

2.2 Memory and Storage Backends

A parallel line of work optimizes WiSARD's memory footprint without modifying the mapping principle. Bloom WiSARD [21] replaces RAM lookup tables with Bloom filters, trading a small false-positive rate for substantial memory savings. ULEEN [24] combines Bloom-filter storage with multi-submodel ensembles and Gaussian thermometer encoding in an architecture targeted at edge inference. LogicWiSARD [4] takes the extreme path: it compiles trained WiSARD RAMs into logic functions, eliminating the lookup table at inference time entirely.

Cuckoo WiSARD [2] introduced cuckoo filters as an alternative to Bloom filters for WiSARD storage, exploiting the lower false-positive rate and constant-time deletion of cuckoo filters [9]. F4RM-Cuckoo, the storage-efficient variant we evaluate in Section 4, is the natural composition of F4RM mapping with this storage backend, demonstrating that mapping rules and storage backends are orthogonal axes of WiSARD optimization. The composability is deliberate: by restricting our contribution to the mapping rule alone, F4RM slots into any of these memory architectures without modification to their respective storage apparatus.

2.3 Ensembling

Ensembling weightless networks has been explored primarily as a route to reducing per-model memory while maintaining accuracy. ULEEN's multi-submodel

architecture [24] trains several smaller WiSARDs with different random permutations and aggregates their responses via summation, the same aggregation rule F4RMS adopts. ClusWiSARD [7] uses a clustering-based ensemble motivated by intra-class heterogeneity rather than ensemble decorrelation. Lusquino [18] surveys WiSARD ensembling broadly, covering bagging, boosting, weighted voting, and other aggregation strategies; sum-of-responses, which F4RMS uses, is one of the established options in this lineage.

The Random Subspace Method [14] is the natural starting point for any ensemble seeking decorrelation via input-view diversity. We tested this for WiSARD and found it consistently weaker: subsets weaken each member faster than decorrelation recovers, a tradeoff that loses for WiSARD members unable to reconstruct missing features [5]. F4RMS instead derives diversity from the mapping randomness F4RM preserves (Section 3.3), keeping every member at full F4RM strength while still drawing the ensemble benefit.

3 F4RM: Feature-Restricted Round-Robin Random Mapping

3.1 Preliminaries: WiSARD and Thermometer Encoding

WiSARD [1] classifies binary inputs through lookup tables called RAMs (Figure 1a). A model with M classes contains M discriminators, one per class; each discriminator holds the same R RAMs, and each RAM stores a counter for every address it has seen. Training is a single pass that increments the correct-class RAM counters; inference queries all discriminators, sums each one's RAM counters, and predicts the highest-scoring class. Real implementations use sparse storage, so memory grows with the number of distinct patterns observed rather than with 2^a [25]. A standard refinement called bleaching [12] suppresses single-example memorization by requiring repeated observation of an address before its counter contributes; we use bleaching threshold $b = 1$ throughout, matching the `wisardpkg` [15] default. The decision pipeline involves no arithmetic beyond integer addition, no floating-point, and no gradient, which is what makes WiSARD attractive at the edge.

Real-valued inputs must be binarized before WiSARD can process them. We use thermometer encoding, the de facto standard in the WiSARD literature. A thermometer encoder of size t maps $x \in [0, 1]$ to the binary vector $[b_1, \ldots, b_t]$ with $b_i = 1$ iff $i/t \leq x$. The encoding preserves a monotonic correspondence between value distance and Hamming distance, two values x_1, x_2 produce codes differing in $t \cdot |x_1 - x_2|$ bits, a property that binary or Gray codes [11] lack. We use the *distributive* variant [17], placing each threshold at a quantile of the training distribution. Features are min-max scaled on the training set before encoding.

An input with n real features at thermometer size t yields a binary vector of length $n \cdot t$, partitioned into $R = \lceil n \cdot t/a \rceil$ disjoint RAM chunks of size a. The assignment of the $n \cdot t$ bits to RAM positions, fixed at initialization, shared across discriminators, is the **mapping** that F4RM redefines.

3.2 Standard Random Mapping and Its Limitations

Standard WiSARD uses a uniformly random permutation as its mapping, conventional, requiring no design effort, and not previously challenged on accuracy grounds. But it treats all $n \cdot t$ bits as interchangeable, ignoring a structural property of thermometer-encoded inputs that can be exploited for free: the t bits of one feature form a contiguous block of 1s followed by 0s, so they take only $t+1$ distinct values instead of 2^t. When random mapping places multiple bits from the same feature into the same RAM, this redundancy imposes two simultaneous costs. *Feature-coverage loss*: a RAM receiving two bits from feature f forces another RAM to receive one fewer, and in the limit leaves some RAM blind to f. *Effective-capacity loss*: c bits from a single feature jointly produce $c+1$ patterns instead of 2^c, shrinking the RAM's usable address space by a factor of $(c+1)/2^c$, down to $3/4$ at $c = 2$ and $1/2$ at $c = 3$.

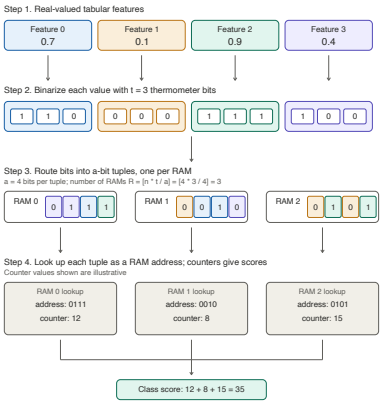
Both costs scale with the frequency of same-feature collisions, governed by the ratio a/n: when $a/n \ll 1$ collisions are rare by chance and both costs are negligible; when $a/n \geq 1$ the pigeonhole principle guarantees at least one collision per RAM, and the expected count grows with the ratio. The ratio a/n is therefore the natural axis along which any mapping-rule benefit must be measured, and it returns throughout the analysis that follows.

3.3 The F4RM Rule

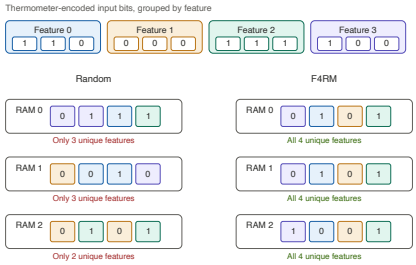
F4RM resolves both costs through a single rule on the mapping: each RAM should read bits from the maximum possible number of distinct features, and when the address size forces some feature to contribute multiple bits to a single RAM, those multi-bit contributions should be spread evenly across RAMs. When $a \leq n$, every RAM reads a distinct features, one bit each; when $a > n$, every RAM reads every feature at least once with the extras spread evenly.

A round-robin construction realizes this rule: at each step it picks the next feature (in a seeded shuffled order), then its next unassigned thermometer bit (in an independently shuffled order), and places the bit in the RAM with the fewest bits from that feature so far (ties broken by RAM load). The construction runs in $O(n \cdot t)$ time, is data-independent and seed-deterministic, and preserves stochastic diversity along the only dimension F4RM does not constrain, thermometer-level placement, so different seeds yield different valid F4RM mappings (exploited by F4RMS in Section 3.5). Figure 1b illustrates the rule on a 4-feature toy example.

Figure 2 shows the same mechanism on real data: F4RM produces visibly uniform feature coverage on Seeds where random mapping clusters same-feature bits (a), and its accuracy advantage on EEG Eye State decays predictably as thermometer size t raises capacity at fixed a, n (b), confirming F4RM improves information efficiency rather than adding capacity, and remaining ahead throughout the sweep.

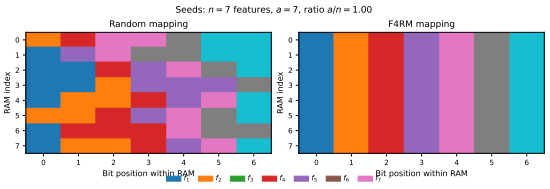


(a) WiSARD pipeline (one discriminator): thermometer-encode features, route bits to fixed-size tuples by the mapping, look up each tuple in its RAM, sum counters for the class score.

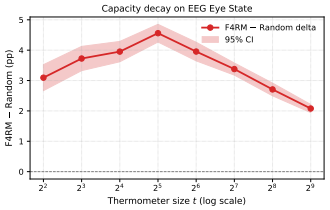


(b) F4RM versus default random mapping on a small example: 12 thermometer bits from 4 features routed into 3 RAMs of size 4. Random clusters bits from the same feature in some RAMs; F4RM guarantees every RAM reads from every feature.

Fig. 1: (a) The WiSARD architecture F4RM operates on. (b) The mapping rule F4RM imposes, contrasted with the standard random permutation.



(a) Random vs. F4RM mapping on Seeds ($n = 7$, $a = 7$, $t = 8$). Random clusters same-feature bits and skips features in some RAMs; F4RM places one bit from every feature in every RAM, producing perfect vertical stripes.



(b) Capacity decay on EEG Eye State: F4RM−Random delta peaks at $t = 32$ (+4.6 pp), decays at higher t.

Fig. 2: F4RM mechanism on real data: uniform feature coverage (a) and a predictable capacity-dependent advantage (b).

3.4 Address-Size Selection

WiSARD's behavior depends strongly on the address size a, and the choice of a has an established trade-off. Too small, and each RAM captures too narrow a bit pattern to support useful discrimination, the model under-fits. Too large, and the nominal address space 2^a exceeds what the training set can populate; most addresses remain empty, and the RAMs memorize training samples rather than generalize [25]. The standard approach in WiSARD work is per-dataset hyperparameter sweep [25], in which a and t are tuned jointly on a validation set to maximize accuracy.

Per-dataset sweeps confound a structural comparison of mappings: selecting hyperparameters per-mapping risks attributing to the mapping what is really a selection effect. To evaluate F4RM against random mapping fairly, we adopt a principled default rule and apply it identically to both:

$$a = \min\left(2n,\ \lfloor \log_2 m \rfloor\right), \tag{1}$$

where n is the number of features and m is the number of training samples. Thermometer size is fixed at $t = 8$ for all datasets.

Each term has an independent justification: $2n$ places $a/n = 2$, within the collision-frequent regime where any mapping-level difference is measurable; $\lfloor \log_2 m \rfloor$ caps the address space below the training-set size, since $2^a > m$ leaves most addresses unreachable and RAMs memorize rather than generalize. The rule is not designed to maximize per-dataset accuracy (grid search yields higher peaks); its purpose is to let F4RM's contribution be evaluated without confounding it with hyperparameter selection. The full per-dataset hyperparameter listing is available in the open-source artifact.

3.5 Compositional Extensions

F4RM is a mapping rule, orthogonal to two other axes along which WiSARD has been refined: ensembling [24,18] and RAM-state storage [21,2,24]. We define two extensions to verify F4RM composes cleanly with both; neither is a novel architecture in itself.

F4RMS (F4RM enSemble) trains K F4RM WiSARDs at identical a and t but different mapping seeds, aggregating their per-class responses by summation as in ULEEN [24]: the ensemble predicts $\hat{y} = \arg\max_c \sum_{k=1}^{K} r_c^{(k)}$. Different seeds produce different valid F4RM mappings (the rule constrains feature placement but leaves thermometer-level placement and feature visit order to randomness), so members differ only in bit-to-RAM assignment. Cost scales linearly with K in raw compute but with no penalty in deployment latency on parallel hardware; Random Subspace Method-style alternatives [14,5] were tested and found weaker (artifact).

F4RM-Cuckoo pairs the F4RM mapping with cuckoo-filter storage [9] in place of the dense per-class counter table, following Cuckoo WiSARD [2] which introduced cuckoo filters as a memory-efficient alternative to Bloom filters [21]. F4RM mapping does not modify how RAM state is stored, only which bits each RAM reads, so the substitution requires no change to the F4RM construction or to inference. We evaluate F4RM-Cuckoo separately because storage choice shifts the memory-accuracy operating point, surfacing the low-memory regime where F4RM's collision-free address utilization matters most.

4 Experimental Methodology

4.1 Datasets and Evaluation Framework

We evaluate on 21 tabular classification datasets drawn from UCI and scikit-learn, with 3 to 40 real-valued features, sample sizes from 150 (Iris) to 20,000

(Letter Recognition), and class counts from 2 to 26. The selection covers standard benchmarks from the WNN literature [25,24] supplemented with additional UCI datasets to broaden coverage of the feature-count and sample-size axes. Per-dataset hyperparameters produced by the principled-default rule (Section 3.4) are documented in the open-source artifact.

Two evaluation frameworks operate over this suite. The first is a matched-hyperparameter protocol comparing F4RM, F4RMS, and the random-mapping baseline at identical (a, t) chosen by the principled-default rule, with statistical significance assessed across 200 paired evaluations per dataset. This protocol isolates the mapping rule's effect from any confounding due to per-method hyperparameter tuning. The second is a Pareto-frontier protocol that sweeps each system across its hyperparameter space, extracts a per-system Pareto frontier on the (memory, accuracy) plane, and compares F4RM-Cuckoo's frontier against those of ten baselines. This second protocol characterizes the operating regimes where each system is competitive. The two protocols together support the paper's two empirical claims: that F4RM improves on random mapping at matched hyperparameters (matched-hyperparameter protocol), and that F4RM-Cuckoo achieves Pareto-frontier membership across diverse memory budgets (Pareto-frontier protocol).

4.2 Matched-Hyperparameter Protocol

For each dataset, we perform 200 independent evaluations. Each evaluation uses a stratified 80/20 train/test split seeded independently per evaluation; preprocessing applies min-max scaling and distributive thermometer thresholds [17] fit on the training fold only, with $t = 8$. All three methods, random-mapped WiSARD, F4RM WiSARD, and F4RMS at multiple values of K, are trained and evaluated on the same split at the address size a produced by Equation 1. Per-method accuracy, training wall-clock time, and per-sample inference wall-clock time are recorded for every evaluation.

Pairing all methods on the same split within each evaluation enables paired statistical tests and removes split-induced variance from method-level comparisons. Across the 200 evaluations, the 200 split seeds are fixed and shared across datasets, so any dataset-level difference in variance is attributable to the dataset rather than to split sampling. F4RM and F4RMS use mapping seeds derived from the evaluation index, ensuring independent mapping draws across the 200 evaluations.

For each pair of methods (F4RM vs Random, F4RMS vs Random, F4RMS vs F4RM), we compute the per-evaluation accuracy delta. We report the mean delta, the Wilcoxon signed-rank test [26] p-value, and a 95% normal-approximation confidence interval on the mean ($\bar{\Delta} \pm 1.96\,\sigma_\Delta/\sqrt{n}$, with $n = 200$). The Wilcoxon test is preferred over the paired t-test because accuracy is bounded in $[0, 1]$ and the delta distribution is rarely Gaussian. We apply Bonferroni correction with family-wise threshold $\alpha = 0.05/21 \approx 0.0024$ across the 21 datasets, and report a comparison as significant only when this corrected threshold is met.

4.3 Pareto-Frontier Protocol

The Pareto-frontier protocol compares F4RM-Cuckoo against ten baselines spanning two families. The first family covers tabular machine learning: Decision Trees (DT) and Random Forests (RF) as the canonical tree-based baselines, gradient-boosted decision tree libraries XGBoost [8], LightGBM [16], and CatBoost [20], multi-layer perceptrons (MLP) as the standard fully-connected neural baseline, and the FT-Transformer architecture [10] as a recent transformer-based tabular model. The second family covers prior weightless neural networks: BTHOWeN's hash-routed Bloom-filter architecture [25], the differentiable WiSARD formulation DWN [3], and ULEEN's multi-submodel ensemble [24]. The tabular ML family represents the strongest non-neural and recent-deep-learning systems available for tabular classification; the prior weightless family represents the strongest direct comparators in our problem class.

For each (system, dataset) pair, we sweep the system's hyperparameter grid and record accuracy, memory footprint, training time, and inference latency for every cell. Per-cell aggregates are taken over 5 random seeds where stochasticity is present (e.g., train/test split, model initialization). We then compute the per-system Pareto frontier on the (memory, accuracy) plane: the set of (memory, accuracy) points such that no other point on that system has both higher accuracy and lower memory. F4RM-Cuckoo's frontier is compared against each baseline's frontier under two criteria: *strict dominance* (every point on the baseline frontier is dominated by some F4RM-Cuckoo point) and *frontier membership* (F4RM-Cuckoo contributes at least one non-dominated point to the joint envelope of both systems' frontiers). The strict-dominance criterion is conservative; the frontier-membership criterion is the natural Pareto-optimality condition.

Memory accounting is system-specific: WiSARD-family systems report cumulative cuckoo-filter or per-class hashmap size; tree-based baselines report native serialized size; neural baselines (MLP, FT-Transformer) report parameter count $\times$ 4 bytes (FP32); prior weightless ports use the original-paper accounting where comparable. The complete hyperparameter grids and memory-accounting rules are documented in the open-source artifact.

4.4 Statistics and Environment

The random-mapping baseline is the standard WiSARD formulation (uniformly random bit-to-RAM permutation); bleaching is enabled at $b = 1$ for all WiSARD-family methods, matching the `wisardpkg` [15] default. F4RM, F4RMS, and F4RM-Cuckoo are implemented on top of `wisardpkg` by supplying an explicit mapping computed by the F4RM round-robin construction; F4RM-Cuckoo additionally substitutes per-class cuckoo filters for the default counter table. All experiments ran on an Apple M4 Pro workstation (24 GB RAM, Python 3.13, CPU only); the matched-hyperparameter protocol completes in approximately one hour and the full Pareto sweep in several hours. All code (including BTHOWeN, DWN, ULEEN ports) is at `https://anonymous.4open.science/r/f4rm-implementation/wisardpkg/`; notebooks at `https://anonymous.4open.science/r/f4rm-notebook/notebooks/f4rm/`.

5 Results

5.1 Matched-Hyperparameter Results

Table 1 reports the headline result: across 21 tabular classification datasets with 200 paired evaluations per dataset and Bonferroni correction at $\alpha = 0.05/21 \approx 0.0024$, F4RM beats random mapping on **13 of 21** datasets with statistical significance and never loses significantly. Across all datasets the mean per-evaluation delta is +1.38 pp; the median is +0.87 pp.

Table 1: Random-mapping vs. F4RM at matched hyperparameters: per-dataset mean test accuracy across 200 paired stratified 80/20 splits, with paired delta in percentage points and Bonferroni-corrected paired Wilcoxon significance flag ($\alpha = 0.05/21 \approx 0.0024$). Best per row bolded; datasets sorted by m.

Dataset	m	n	a	Random	F4RM	Δ (pp)	Sig.
Iris	150	4	7	0.931	**0.935**	+0.42	,
Wine	178	13	7	0.966	**0.967**	+0.10	,
Seeds	210	7	7	0.890	**0.899**	+0.87	,
Glass	214	9	7	0.692	**0.702**	+0.93	,
Haberman	306	3	6	**0.732**	0.729	−0.31	,
Ecoli	336	7	8	0.803	**0.815**	+1.16	✓
Ionosphere	351	34	8	**0.905**	0.897	−0.77	,
Breast Cancer	569	30	9	0.960	**0.962**	+0.18	,
Balance Scale	625	4	8	0.625	**0.702**	+7.73	✓
Pima Diabetes	768	8	9	0.698	**0.705**	+0.74	,
Vehicle	846	18	9	0.674	**0.687**	+1.35	✓
Banknote	1372	4	8	0.918	**0.948**	+2.96	✓
Yeast	1484	8	10	0.451	**0.470**	+1.89	✓
Steel Plates	1941	33	10	0.852	**0.871**	+1.95	✓
Segment	2310	19	11	0.931	**0.936**	+0.43	✓
Waveform	5000	40	12	0.774	**0.778**	+0.47	✓
SatImage	6430	36	12	0.828	**0.833**	+0.44	✓
Pendigits	10992	16	13	0.960	**0.968**	+0.77	✓
EEG Eye State	14980	14	13	0.699	**0.738**	+3.89	✓
Magic Gamma	19020	10	14	0.766	**0.783**	+1.68	✓
Letter Recognition	20000	16	14	0.854	**0.875**	+2.11	✓
Significant (Bonferroni):							13/21

The pattern matches the mechanism of Section 3.2. F4RM's largest gains concentrate on datasets where each feature carries non-redundant signal (EEG, Magic Gamma, Yeast, Letter Recognition, Pendigits); its smallest gains are on datasets with internally-redundant feature sets, Ionosphere (complex-valued pairs from 17 pulse numbers [22]) and Breast Cancer (mean/std-error/"worst" variants of 10 underlying measurements [23]), where the coverage guarantee has little to redistribute. The three regimes are dense-signal (largest gains), redundant-feature (small gains), and small-sample (variance swamps sub-pp effects under the $\lfloor \log_2 m \rfloor$ cap).

5.2 F4RMS Extension

The F4RMS ensemble (Table 2) extends F4RM's gains: at $K = 50$, F4RMS beats random mapping on **20 of 21** datasets and beats single F4RM on **20 of 21** with Bonferroni significance, including recoveries where single F4RM was neutral (Ionosphere +3.37 pp, Breast Cancer +0.68 pp). The single exception is Haberman, where F4RMS loses significantly to both Random (−3.88 pp) and F4RM (−3.57 pp): with only three features, F4RM constrains placement so strongly that mapping seeds produce nearly identical mappings, a mechanism-supportive boundary case where the ensemble has no diversity to exploit.

Table 2: F4RMS extension at $K = 50$: per-dataset accuracy, paired delta vs. Random in pp, and Bonferroni-corrected significance (which matches vs. Random and vs. F4RM on every dataset; only Haberman is ,). Datasets sorted by m.

Dataset	Acc.	Δ	Sig.	Dataset	Acc.	Δ	Sig.	Dataset	Acc.	Δ	Sig.
Iris	0.957	+2.58	✓	Pima Diabetes	0.744	+4.55	✓	SatImage	0.895	+6.72	✓
Wine	0.979	+1.29	✓	Vehicle	0.728	+5.40	✓	Pendigits	0.990	+2.95	✓
Seeds	0.914	+2.44	✓	Banknote	0.997	+7.87	✓	EEG Eye State	0.907	+20.82	✓
Glass	0.738	+4.52	✓	Yeast	0.589	+13.77	✓	Magic Gamma	0.860	+9.34	✓
Haberman	0.693	−3.88	,	Steel Plates	0.976	+12.41	✓	Letter Recognition	0.948	+9.39	✓
Ecoli	0.858	+5.51	✓	Segment	0.963	+3.18	✓	**Sig. Bonferroni: 20/21**			
Ionosphere	0.939	+3.37	✓	Waveform	0.847	+7.35	✓				
Breast Cancer	0.967	+0.68	✓								
Balance Scale	0.847	+22.25	✓								

5.3 Pareto-Frontier Results

Across the Pareto-frontier protocol, F4RM-Cuckoo achieves Pareto-frontier membership on every one of the 21 datasets against every one of the 10 baselines: on no (dataset, baseline) pair does F4RM-Cuckoo's frontier sit entirely below the baseline's; it always contributes at least one non-dominated point to the joint envelope. Table 3 reports the dominance counts aggregated across all 21 datasets; per-dataset frontier plots are in the open-source artifact.

A stronger framing: F4RM-Cuckoo achieves a higher accuracy-per-byte ratio than every gradient-boosting baseline (XGBoost, LightGBM, CatBoost) and random forest on **every** one of 21 datasets, plus 21/21 vs. FT-Transformer and 18/21 vs. MLP among deep tabular baselines, and 19/21 vs. DWN among prior weightless systems. Stratifying by deployment memory budget, F4RM-Cuckoo wins the majority of comparable datasets in the 10–100 KB and 1–10 KB bands typical of microcontroller flash and SRAM at the edge, including 14/14 against BTHOWeN in 10–100 KB; below 1 KB, BTHOWeN wins on 12/20 (the cuckoo filter is too small for F4RM's bit redistribution to express). The comparison is closer against ULEEN (9/21 accuracy-per-byte) and BTHOWeN (6/21) at unconstrained budgets, where the architectures are designed for similar memory regimes.

6 Limitations, Conclusion, and Future Work

F4RM is positioned for tabular classification with thermometer-encoded continuous features and a high address-to-feature ratio a/n; on image data ($a/n \ll 1$)

Table 3: Pareto-dominance counts of F4RM-Cuckoo against each of the ten baselines, aggregated across all 21 datasets. *Strict-dom.*: every point on the baseline's frontier is dominated by some F4RM-Cuckoo point. *Mixed*: both systems contribute non-dominated points to the joint envelope. *Env. share*: mean fraction of joint Pareto-envelope points contributed by F4RM-Cuckoo. F4RM-Cuckoo is never strictly dominated by any baseline on any dataset (omitted column, always 0/21).

Baseline	Strict-dom. (of 21)	Mixed (of 21)	Env. share
DT	11	10	74.1%
RF	9	12	89.3%
XGBoost	6	15	89.0%
LightGBM	6	15	83.6%
CatBoost	3	18	90.3%
MLP	14	7	90.5%
FT-Transformer	18	3	**98.5%**
BTHOWeN	1	20	72.1%
DWN	12	9	96.4%
ULEEN	4	17	77.5%

collisions are rare by chance and F4RM has nothing to redistribute. F4RMS inherits the same scope condition plus a stricter one: at very small feature counts (e.g., Haberman with $n = 3$) different mapping seeds produce nearly identical mappings, and the ensemble cannot exploit diversity it does not have. F4RM increases memory by 3–18% at matched hyperparameters compared to random mapping, a consequence of populating addresses random mapping leaves unused, and the principled-default address-size rule is designed for rigor of comparison, not per-dataset accuracy. Our reference ports of BTHOWeN, DWN, and ULEEN reproduce paper-reported accuracy within 2–3 pp on most datasets, with two documented gaps; the open-source artifact details these.

The paper presented F4RM, a heuristic feature-coverage mapping rule for WiSARD that resolves two costs random mapping silently imposes on thermometer-encoded inputs. F4RM is parameter-free, runs once at initialization in $O(n \cdot t)$ time, and adds no training or inference compute. Two compositional extensions, F4RMS (ensemble) and F4RM-Cuckoo (cuckoo-filter storage), demonstrate F4RM's compatibility with established WiSARD techniques without modification. Across 21 tabular datasets, F4RM beats random mapping at matched hyperparameters on 13 of 21 with Bonferroni-corrected significance and never significantly loses; F4RMS extends this to 20 of 21; and F4RM-Cuckoo achieves Pareto-frontier membership on every one of 21 datasets against every one of 10 baselines, with a higher accuracy-per-byte ratio than every gradient-boosting baseline and random forest across all 21 datasets. The bit-to-RAM mapping in WiSARD, defaulted to a uniformly random permutation for four decades, was a structural component left undesigned; F4RM is one instance of replacing it with a redundancy-aware rule.

Future work includes a spatially-aware variant for image data (where patches replace features as the unit F4RM redistributes), F4RM-BTHOWeN and F4RM-DWN compositions that this paper has not evaluated, heterogeneous-(a, t) F4RMS in the spirit of ULEEN [24], and analytical bounds linking expected F4RM gain to measurable dataset properties.

Acknowledgments. The authors used Claude Opus 4.7 (Anthropic) as a research assistant for code development, experimental analysis, and manuscript preparation.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Aleksander, I., Thomas, W.V., Bowden, P.A.: WISARD: a radical step forward in image recognition. Sensor Review **4**(3), 120–124 (1984). https://doi.org/10.1108/eb007637
2. Santiago de Araújo, L.: On Optimization of Hardware-Assisted Security and Weightless Neural Network Memory Footprint. Ph.D. thesis, Universidade Federal do Rio de Janeiro (COPPE/PESC) (2019), https://pesc.coppe.ufrj.br/uploadfile/publicacao/2912.pdf
3. Bacellar, A.T.L., Susskind, Z., Breternitz Jr., M., John, E., John, L.K., Lima, P.M.V., França, F.M.G., Miranda, I.D.S.: Differentiable weightless neural networks. In: Proceedings of the 41st International Conference on Machine Learning (ICML). Proceedings of Machine Learning Research, vol. 235, pp. 2277–2295. PMLR (2024)
4. Bacellar, A.T.L., Susskind, Z., Villon, L.A.Q., Miranda, I.D.S., Santiago, L., Dutra, D.L.C., Lima, P.M.V., França, F.M.G., John, L.K., Breternitz Jr., M.: LogicWiSARD: Memoryless synthesis of weightless neural networks. In: Proceedings of the IEEE International Conference on Application-specific Systems, Architectures and Processors (ASAP). pp. 19–26 (2022). https://doi.org/10.1109/ASAP54787.2022.00014
5. Breiman, L.: Random forests. Machine Learning **45**(1), 5–32 (2001). https://doi.org/10.1023/A:1010933404324
6. Capogrosso, L., Cunico, F., Cheng, D.S., Fummi, F., Cristani, M.: A machine learning-oriented survey on tiny machine learning. IEEE Access **12**, 23406–23426 (2024). https://doi.org/10.1109/ACCESS.2024.3365349
7. Cardoso, D.O., Lima, P.M.V., De Gregorio, M., Gama, J.a., França, F.M.G.: Clustering data streams with weightless neural networks. Neurocomputing **183**, 126–135 (2017)
8. Chen, T., Guestrin, C.: XGBoost: A scalable tree boosting system. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD). pp. 785–794. ACM (2016). https://doi.org/10.1145/2939672.2939785
9. Fan, B., Andersen, D.G., Kaminsky, M., Mitzenmacher, M.D.: Cuckoo filter: Practically better than Bloom. In: Proceedings of the 10th ACM International Conference on Emerging Networking Experiments and Technologies (CoNEXT). pp. 75–88 (2014). https://doi.org/10.1145/2674005.2674994
10. Gorishniy, Y., Rubachev, I., Khrulkov, V., Babenko, A.: Revisiting deep learning models for tabular data. In: Advances in Neural Information Processing Systems (NeurIPS) 34. pp. 18932–18943. Curran Associates (2021)
11. Gray, F.: Pulse code communication (1953), u.S. Patent 2,632,058, filed November 13, 1947, issued March 17, 1953
12. Grieco, B.P.A., Lima, P.M.V., De Gregorio, M., França, F.M.G.: Producing pattern examples from "mental" images. Neurocomputing **73**(7–9), 1057–1064 (2010). https://doi.org/10.1016/j.neucom.2009.11.015

13. Guarisa, G.P.: Estudo Comparativo de Técnicas de Mapeamento Binário para Redes Neurais Sem Peso WiSARD. M.Sc. dissertation, Universidade Federal do Rio de Janeiro (COPPE/PESC) (2020), https://pesc.coppe.ufrj.br/uploadfile/publicacao/2973.pdf
14. Ho, T.K.: The random subspace method for constructing decision forests. IEEE Transactions on Pattern Analysis and Machine Intelligence **20**(8), 832–844 (1998). https://doi.org/10.1109/34.709601
15. IAZero: wisardpkg: A library for implementation of weightless neural networks. https://github.com/IAZero/wisardpkg (2018)
16. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., Liu, T.Y.: LightGBM: A highly efficient gradient boosting decision tree. In: Advances in Neural Information Processing Systems (NeurIPS) 30. pp. 3146–3154. Curran Associates (2017)
17. Lima, P.M.V., de Leon, A.F.R.A., França, F.M.G.: Distributive thermometer: A new unary encoding for weightless neural networks. In: Proceedings of the European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning (ESANN) (2020)
18. Lusquino Filho, L.A.D.: Extending WiSARD to Perform Ensemble Learning, Regression, Multi-label, and Multi-modal Tasks. Ph.D. thesis, Universidade Federal do Rio de Janeiro (COPPE/PESC) (2021), https://pesc.coppe.ufrj.br/uploadfile/publicacao/3004.pdf
19. Menghani, G.: Efficient deep learning: A survey on making deep learning models smaller, faster, and better. CoRR **abs/2106.08962** (2021), https://arxiv.org/abs/2106.08962
20. Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A.V., Gulin, A.: CatBoost: Unbiased boosting with categorical features. In: Advances in Neural Information Processing Systems (NeurIPS) 31. pp. 6638–6648. Curran Associates (2018)
21. Santiago, L., Verona, L., Rangel, F., Firmino, F., Lima, P.M.V., Menasche, D., Caarls, W., Breternitz Jr., M., Lengerke, O., de Araujo, L.S., França, F.M.G.: Weightless neural networks as memory segmented Bloom filters. Neurocomputing **416**, 272–280 (2020). https://doi.org/10.1016/j.neucom.2019.11.095
22. Sigillito, V.G., Wing, S.P., Hutton, L.V., Baker, K.B.: Classification of radar returns from the ionosphere using neural networks. Johns Hopkins APL Technical Digest **10**(3), 262–266 (1989)
23. Street, W.N., Wolberg, W.H., Mangasarian, O.L.: Nuclear feature extraction for breast tumor diagnosis. In: Proceedings of the IS&T/SPIE International Symposium on Electronic Imaging: Science and Technology. vol. 1905, pp. 861–870. SPIE (1993)
24. Susskind, Z., Arora, A., Miranda, I.D.S., Bacellar, A.T.L., Villon, L.A.Q., Katopodis, R.F., de Araujo, L.S., Dutra, D.L.C., Lima, P.M.V., França, F.M.G., Breternitz Jr., M., John, L.K.: ULEEN: A novel architecture for ultra-low-energy edge neural networks. ACM Transactions on Architecture and Code Optimization (TACO) **20**(4) (2023). https://doi.org/10.1145/3629522
25. Susskind, Z., Arora, A., Miranda, I.D.S., Villon, L.A.Q., Katopodis, R.F., de Araujo, L.S., Dutra, D.L.C., Lima, P.M.V., França, F.M.G., Breternitz Jr., M., John, L.K.: Weightless neural networks for efficient edge inference. In: Proceedings of the International Conference on Parallel Architectures and Compilation Techniques (PACT). pp. 279–290 (2022). https://doi.org/10.1145/3559009.3569680
26. Wilcoxon, F.: Individual comparisons by ranking methods. Biometrics Bulletin **1**(6), 80–83 (1945). https://doi.org/10.2307/3001968